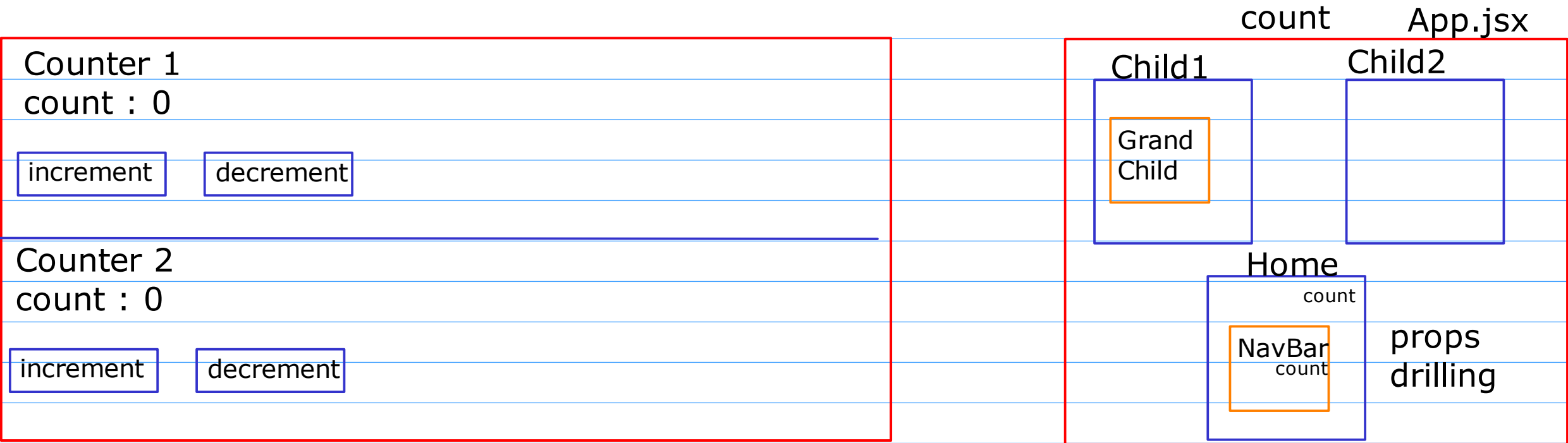
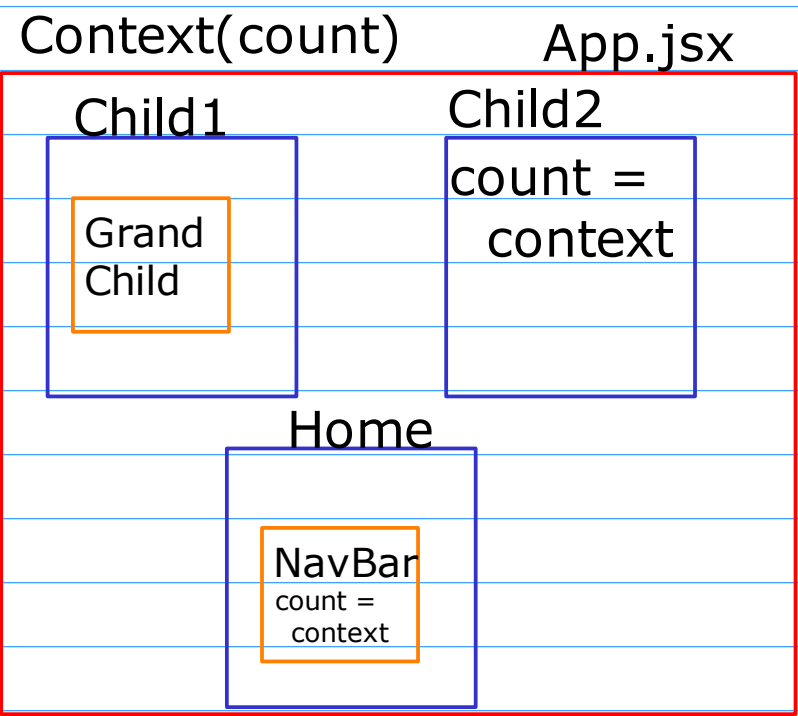




- # Property drilling
- Passing the state from the parent component to the Child component and the child component passing the same property to the grandchild component is called as property drilling
 - If the Child component is not consuming the state and passing directly to the grandchild then the state passed in the child component is useless
 - to avoid such props drilling we should use context api

- # Context Api
- It is used to share the state created in parent component to all its child and grand child components without props drilling



/

/register

/home

/home/food

/home/cart

/home/orders

/home/profile

class Employee{

Employee(){}

~Employee(){}

}

class Person{

Person()

~Person()

}

class Date{

Date()

~Date()

}

/courses
get
post
put
delete

Course.jsx
AddCourse.jsx

Course.jsx

AddCourse

id

del

--	--	--

Add

Edit

id

price

Courses

Redux

- Global State Managment
- Basic Terminologies of Redux
 - 1. Reducers
 - This is the function that accepts initial state and an action object and returns a new state
 - 2. action
 - It is an object that has 2 properties
 - a. type
 - b. payload

App.jsx

Home.jsx

Counter1

createContext()

Counter2

Counter3

count : {count}

increment

decrement

count : {count}

increment

decrement

count : {count}

increment

decrement

count -> increment,decrement
cart -> Add items in to the cart
remove items from the cart
update the qty in the cart

```
const [cart,setCart] = useState([])  
const arr = []  
arr.push({f: 1,name:'pavbhaji'})  
setCart(arr)  
  
setCart([...arr,{}])
```

Why Redux?

- It manages the state globally
- It performs the changes in the state by taking an action from us

eg : Counter Reducer => Initial State => {count : 0}

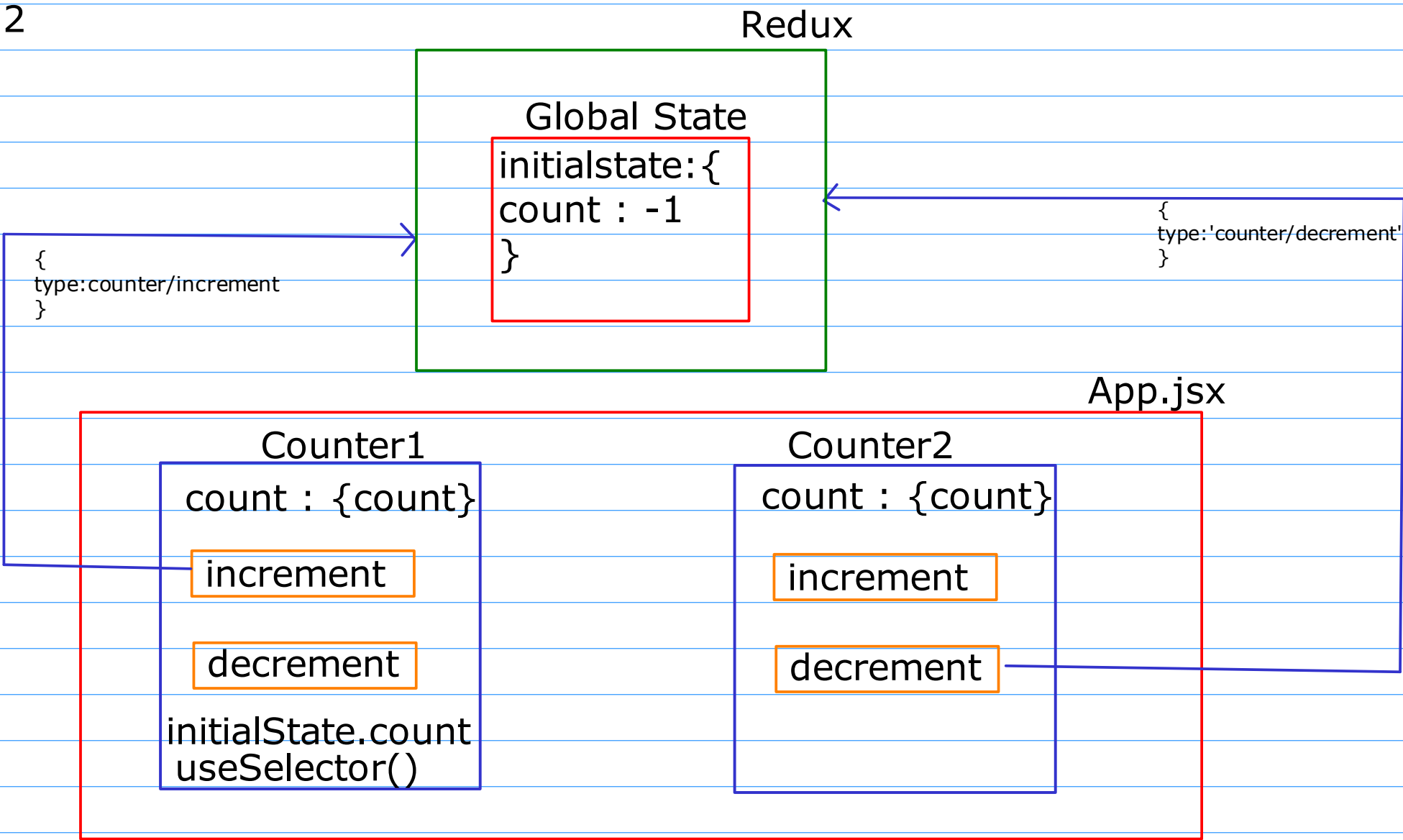
```
CounterReducer()  
function reducer(initialState={count : 0},action){  
  const newState = {...initialState}  
  if(action.type == 'counter/increment'){  
    newState.count+=1  
  }  
  else if(action.type == 'counter/decrement'){  
    newState.count-=action.payload  
  }  
  
  return newState  
}
```

It will return a new state based on initial state and action

```
const initialState1 = reducer(); // count:0  
const initialState2 = reducer(initialState,a1)// count:1  
const initialState2 = reducer(initialState2,a1)// count:2  
  
const a1 = {  
  type: 'counter/increment',  
  payload:undefined  
}
```

useDispatch()
- it is used to create the action object

```
const a2 = {  
  type:'counter/decrement',  
  payload : 2  
}
```



Store

Global State

Reducer functions

slice	slice	slice
counter { count:0 }	user { user:null }	cart { cart:[] }

CounterSlice

```
const counterSlice = createSlice({
  name: counter,
  initialState : {
    count :0
  },
  reducers:{
    increment :(state,action)=>{
      state.count+=1
    } ,
    decrement : (state,action)=>{
      state.count-=action.payload
    }
  }
})
```